Documentação Trabalho Prático 1

Algoritmos II

Vitor Terra Mattos do Patrocínio Veloso - 2022035741

Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte - MG - Brasil

vtmpv2210@gmail.com

1. Introdução

O propósito deste trabalho foi desenvolver um sistema capaz de, a partir de uma frota de motoristas (a princípio gerada aleatóriamente), discernir quais motoristas pertencentes à frota estariam até a uma certa distância de um restaurante em Belo Horizonte, assim como calcular o tempo estimado de chegada de cada motorista em distância hábil até o restaurante.

O sistema deveria armazenar os motoristas da frota em uma Trie Compacta, baseando-se no geohash resultante da conversão da longitude e latitude de cada um deles, a fim de agilizar consideravelmente o processo de busca por motoristas hábeis (não sendo necessário calcular a distância até o restaurante de cada motorista gerado)

O sistema final conta também com um filtro de tempo máximo de chegada até o restaurante. Este filtro tem o propósito de minimizar o tempo de busca por um caminho no grafo e pode ser deixado vazio para que não haja limite.

Para utilizar o programa, instale as bibliotecas especificadas em “requirements.txt” usando o comando “pip install -r requirements.txt”, usando ambiente virtual caso desejado, em seguida use o comando “python UI.py” para inciar a interface gráfica. Nas caixas indicadas escolha o número de motoristas para se gerar, o restaurante desejado e a distância máxima desejada entre os motoristas e o restaurante. Adicionalmente pode-se escolher um tempo máximo esperado de chegada até o restaurante. Isto não vai interferir na busca, porém pode reduzir o tempo levado na exploração do grafo.

2. Metodologia

2.1 Configurações

O código foi desenvolvido em Python 3.13.14

Sistema Operacional: Windows 10.
Processador: Intel(R) Core(TM) I5-12400F @ 2.50 GHz.
RAM: 32 GB.

2.2 Bibliotecas Usadas

- Dash e Dash-Leaflet foram usadas para fazer a interface gráfica.
- OSMnx foi usada para gerar o grafo de Belo Horizonte utilizado, bem como manipulá-lo em alguns pontos.
- Networkx foi usada para calcular o algoritmo de Dijkstra no grafo gerado através da OSMnx, para encontrar o menor caminho entre motoristas e restaurantes. (Especificamente, encontra-se um caminho do restaurante até os motoristas utilizando o grafo inverso, gerado pela OSMnx).
- Pyproj foi usada para fazer a conversão entre as posições no formato utilizado pelos nossos dados e o sistema de longitudes e latitudes (mais detalhes no item 2.2).
- Haversine foi usada para calcular a distância de haversine entre pontos de longitude e latitude no mapa (especificamente a distância entre os restaurantes e os motoristas).
- Scikit-Learn foi instalada como dependência da OSMnx.

2.3 Testes e Experimentos

A grande maioria dos testes foram feitos através de breakpoints() em pontos estratégicos do código. Na função ‘main’ do arquivo trie.py foi deixado um breakpoint caso seja desejado testar o funcionamento da árvore. Para tal, simplesmente execute “python trie.py” após instalar as devidas bibliotecas.

Como será explicado adiante, também foi deixado no arquivo “geo.py” o script experimental usado para colher dados através de uma biblioteca de geolocalização. Como este código não foi usado para o projeto final, ele foi deixado no arquivo apenas como comentário. Caso queira rodar o código, remova os símbolos “#” na área indicada e execute “python geo.py” após instalar as devidas bibliotecas (note que a biblioteca de geolocalização não foi colocada como dependência do projeto, portanto deve ser instalada manualmente). Deve-se notar antes de rodar este código, entretanto, que ele leva cerca de oito horas para rodar em sua completude e pode produzir resultados diferentes dos que foram encontrados inicialmente.

2.4 Coleta de Dados

Inicialmente se fez necessário obter os dados que seriam usados para localizar os restaurantes. Como a planilha fornecida no enunciado do trabalho não consta com dados de localização (além do endereço escrito por extenso), à recomendação do

professor, fez-se uso de uma planilha alternativa, encontrada no Portal de Dados Abertos da prefeitura de Belo Horizonte. A planilha exata da qual os dados usados foram retirados está disponível neste link. A versão curada dela para exibir apenas restaurantes e bares está disponível para download na página GitHub deste projeto sobre o nome de “atividades_economicas_curado.csv”. O processo exato através do qual apenas os dados necessários foram extraídos da planilha original será detalhado a seguir.

Como a nova planilha continha dados não só de restaurantes e bares, mas de todo tipo de negócio em Belo Horizonte, da planilha original (“bares.csv”), extraiu-se um conjunto de todos os elementos diferentes que apareciam na coluna “CNAE” (Classificação Nacional de Atividades Econômicas), em seguida, da nova planilha, extraiu-se apenas as colunas relevantes dos elementos cujos quais a coluna “CNAE” coincide com algum valor de CNAE presente em “bares.csv”. Especificamente, as colunas retiradas foram “_id”, “NOME”, “NOME_FANTASIA”, “GEOMETRIA”, “NOME_BAIRRO”, “DESC_LOGRADOURO”, “NOME_LOGRADOURO”, “NUMERO_IMOVEL” e “COMPLEMENTO”. No projeto final, as colunas de nome e nome fantasia, bem como as referentes ao endereço do estabelecimento foram usadas para que se pudesse realizar a busca pelo restaurante no mapa interativo através destes componentes. A coluna de geometria foi utilizada para fazer a geolocalização dos restaurantes através do uso de um conversor da biblioteca externa “*pyproj*”, que foi usada para traduzir os pontos em WKT (Well-Known Text) no formato SIRGAS 2000 / UTM zone 23S (EPSG:31983) para suas longitudes e latitudes. A coluna “_id” acabou por não ser utilizada, pois foi mais conveniente dar ids novos aos restaurantes começando de 0 até 20949, já que os items originais referentes aos demais ids não seriam utilizados.

Vale à pena fazer menção de uma tentativa inicial de utilizar uma biblioteca de geolocalização (*Nominatim*) para obter os dados de localização dos restaurantes a partir de seus endereços na planilha original. Optou-se por não seguir por esta rota no final tanto pela qualidade dos dados que se mostrou muito inferior, por vezes a biblioteca sendo incapaz de encontrar endereços além do logradouro, o que fazia com que alguns restaurantes na mesma rua aparecessem no mesmo lugar e por outras falhando completamente em encontrar a localização desejada. O código utilizado nesta tentativa foi mantido no projeto na forma de comentários, caso a visualização dele seja desejada. Os dados obtidos por este meio não serão disponibilizados na página GitHub a fim de evitar a poluição do projeto, porém estão disponíveis para download nesta pasta do Google Drive, caso os resultados sejam de interesse.

2.5 Implementação

- **Geohash**
  A implementação do sistema de geohashes se deu através uma função que recebe coordenadas em longitude e latitude e as converte para seus respectivos geohashes de oito caracteres. Os geohashes são gerados da seguinte forma:

Primeiro, inicializamos um número binário que vai se referir à longitude. Toma-se então a longitude e verifica-se em qual parte do intervalo]-180, 180] ela se encontra; se ela for maior ou igual à metade do intervalo (neste caso 0), o primeiro dígito do número binário que estamos gerando será 1, do contrário, será 0. Atualizamos então o intervalo com um novo limite à esquerda, média e limite à direita.

Se o valor da longitude estiver à direita do intervalo, os novos limites serão:
[Antiga Média **|** (Antiga Média + Limite à Direita)/2 **|** Limite à Direita (mantido)]

Se estiver à esquerda:
[Limite à Esquerda(mantido) **|** (Limite à Esquerda + Antiga Média)/2 **|** Antiga Média]

O processo é então repetido ao longo de no máximo 32 bits para gerar o valor binário que irá representar a longitude. A mesma coisa é então feita para a latitude, com a exceção de que a latitude usa o intervalo]-90, 90].

Após obter os números referentes à longitude e à latitude, uma função ajudante é chamada para intercalar seus dígitos, formando um número de no máximo 64 bits (por isso os números de longitude e latitude devem ter no máximo 32). Isso significa que o novo número que vamos formar terá [bit1_longitude | bit1_latitude | bit2longitude | bit2latitude, (…)].

Em seguida, separamos o novo número gerado em intervalos de cinco bits, que é a quantidade necessária para representar qualquer um dos 32 valores no alfabeto de geohash. Tomamos então os valores correspondentes no alfabeto geohash aos números obtidos pelos primeiros oito intervalos de 5 bits e os concatenamos para formar o geohash final que será usado.

- **Trie**

A classe Trie foi responsável por gerar, armazenar e buscar os dados referentes aos motoristas. Ela é suportada pela classe Node, que por sua parte possui três atributos:

1. Geohash: O geohash armazenado naquele nó. Não precisa ser um geohash completo; por exemplo, para um geohash "7h2wqz3g", “72h” pode estar em um filho do nó raiz, ”w” em um filho subsequente, “q” em outro filho subsequente e “z3g” no nó folha.
2. Children: Um dicionário que mantém referência aos filhos do nó. Mapeia-se {“primeiro caractere do atributo geohash do filho” : “o nó filho em si”}.

3. Drivers: Os ids de cada motorista armazenado naquele nó. Como cada célula de geohash representa uma área e motoristas são gerados em volta do mesmo ponto, frequentemente vários motoristas acabam tendo o mesmo geohash.

- **Geração de Motoristas**

  Toda vez que são gerados novos motoristas, a Trie é efetivamente reiniciada. Isto foi implementado como comportamento da própria função “generate_drivers()”.

  São gerados a quantidade especificada de motoristas e à cada motorista é atribuído um bar. Caso existam mais motoristas do que restaurantes, cada restaurante automaticamente ganha um motorista. Isso ocorre até que existam mais bares do que motoristas, a que ponto, o restante dos motoristas são distribuídos aleatoriamente entre os restaurantes. Como existem cerca de 30000 restaurantes no nosso conjunto de dados entretanto, dificilmente é razoável existirem mais motoristas que restaurantes.

  Em seguida, cada motorista recebe um deslocamento aleatório de seu restaurante de origem (gerado por “random.gauss(0, sigma)”). Para calcular a posição do motorista pré e pós deslocamento, usamos o mesmo sistema da coluna “GEOMETRIA” do nosso conjunto de dados, de forma que possamos simplesmente adicionar o número gerado às coordenadas do motorista.

  Enfim, convertemos as coordenadas geradas para longitude e latitude e usamos “ox.distance.nearest_nodes(GRAFO, X=driver_lons, Y=driver_lats)” para mapear as coordenadas geradas para a via mais próxima. Os dados de longitude, latitude, geometria, geohash, bem como o nó referente ao motorista são armazenados em um dicionário que é então colocado em uma lista que eventualmente é retornada pela função “generate_drivers()”. Assim, o id de cada motorista é a posição de seu dicionário nesta lista

- **Armazenamento**

  A inserção de motoristas na Trie ocorre da seguinte maneira:

A função recebe o nó a partir de qual começar sua busca, o id do motorista (sua posição na lista mencionada no item anterior) e a string representativa de seu geohash

Verifica-se se existe um filho do nó atual referente à letra inicial da string de geohash que se está tentando inserir.

Caso não exista, inserimos o filho no nó atual.

Caso exista, verificamos se o geohash que temos é igual ao geohash do filho, caso seja, inserimos o motorista atual diretamente no nó que se segue. Isto é possível pois nossos geohashes sempre têm exatamente oito caracteres de tamanho (nunca mais ou menos).

Caso os geohashes não sejam iguais, verificamos o maior prefixo comum entre o geohash que temos atualmente e o geohash armazenado no filho que estamos analisando.

Se o maior prefixo tiver o mesmo tamanho do geohash armazenado no filho, chamamos a função recursivamente iniciando do nó filho com o nosso geohash truncado, tendo início a partir da parte que não é igual ao geohash do filho.

Se o maior prefixo for menor do que o geohash armazenado no filho, “quebramos” o nó, partindo o geohash armazenado lá na parte onde ele se diferencia do geohash que queremos inserir e criando dois novos filhos para ele. Um dos filhos herda o restante do geohash original, bem como seus antigos filhos ou ids de motoristas, enquanto o outro fica com o restante do novo geohash e o id do novo motorista.

- **Busca**

  A busca por motoristas tem início limitando a área em que motoristas podem ser achados. Isto é feito através de essencialmente uma tabela, que converte distâncias em metros para o tamanho mínimo de uma célula de geohash (quantos caracteres devem ser usados) que cobre aquela distância. Em seguida, criamos uma lista contendo a célula encontrada, bem como as 8 células adjacentes e criamos uma lista com os ids de todos os motoristas que estão dentro daquela célula. Em seguida, calculamos a distância de fato até o restaurante de cada motorista encontrado e retornamos apenas os ids daqueles que realmente satisfazem os requisitos da busca.

- **Interface Gráfica**

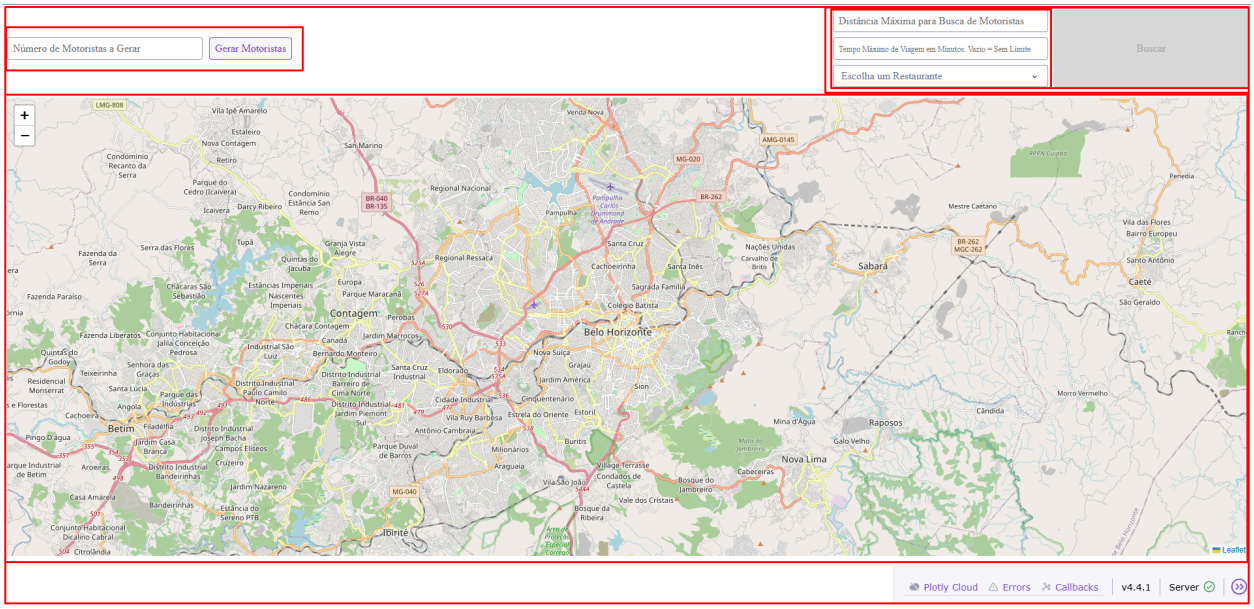
Finalmente a interface gráfica, como sugerido no enunciado deste projeto foi feita utilizando o framework Dash para o *frontend* enquanto o backend foi feito através dos demais artifícios mencionados até agora. O uso pretendido do programa envolve rodar o arquivo python “UI.py”, onde foi feita a implementação da interface gráfica.

Sua execução começa por transformar o arquivo contendo os dados utilizados em um dicionário com {id : {informações sobre o restaurante “id”}}, ou seja, um dicionário de dicionários com informações sobre cada restaurante analisado. Ele é mantido como uma variável global do programa.

Em seguida, carregamos o grafo de Belo Horizonte e adicionamos velocidades e tempos de viagem às suas arestas. Isto é feito através da biblioteca OSMnx que foi usada também para gerar o grafo. Criamos também uma versão inversa do grafo, que será usada para calcular as distâncias entre os restaurantes e os motoristas, pois assim podemos usar o algoritmo de Dijkstra (implementado pela biblioteca Networkx) para encontrar os caminhos ideais apenas uma vez, ao invés de termos de encontrar os caminhos individuais entre cada motorista e o restaurante. Tanto o grafo quanto o grafo inverso são mantidos como variáveis globais do programa.

Em seguida criamos a Trie que vai ser usada para armazenar, gerar e buscar os motoristas, como discutido anteriormente, e inicializamos a lista que vai manter as informações de todos os motoristas que serão gerados. Estas também são mantidas como variáveis globais.

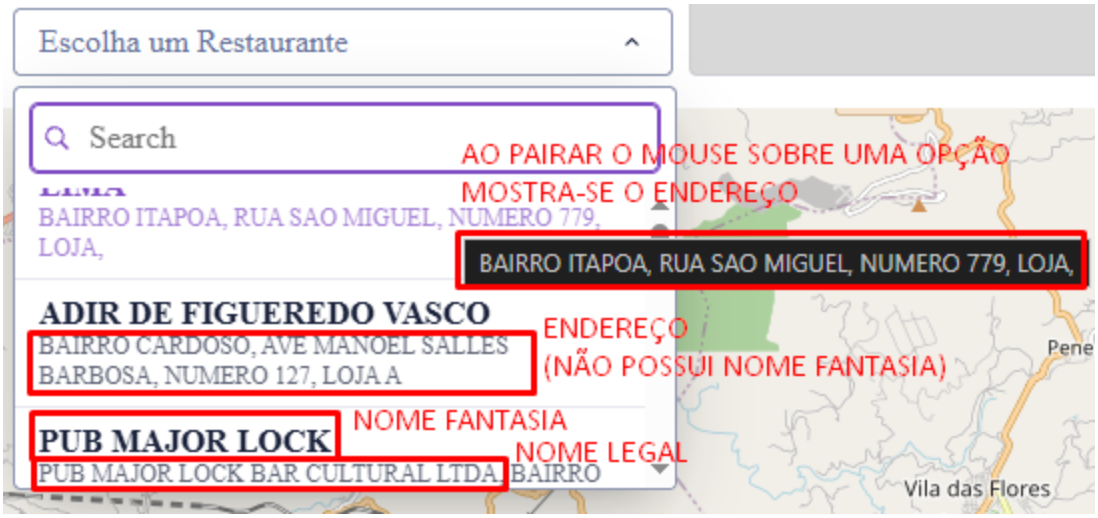
A interface em si é composta por um Div onde está toda a página gerada e que é composto de outros Divs menores: Um para a caixa de texto referente ao número de motoristas que se deseja gerar, bem como o botão que os gera; um para o Div que contém a caixa de texto para distância máxima de busca, o tempo máximo de viagem e o menu dropdown com os restaurantes e para o Div que contém o botão de busca e outro para o mapa de Belo Horizonte, que é um mapa centralizado no ponto [-19.92 Latitude, -43.94 Longitude]. Abaixo segue uma imagem da interface e suas subdivisões.



Em seguida, uma descrição das subrotinas que gerem a página gerada.

Ao apertar o botão de “Gerar Motoristas”, o valor na caixa de texto ao lado do botão é lido e a Trie é usada para gerar os motoristas como discutido anteriormente; é então gerado um marcador vermelho para cada motorista gerado e a lista de marcadores é retornada para o mapa, onde eles são exibidos. O botão pode ser clicado sem que se escolha um número de motoristas; nenhum marcador é gerado e o mapa continua vazio.

O dropdown onde se pode buscar e escolher um restaurante tem suas opções geradas pela função “make_search_options()”; para cada restaurante no dicionário global mencionado anteriormente, ela cria dois Divs, um contendo o nome fantasia do restaurante (ou nome legal, se não houver) e outro contendo o nome legal (se existia nome fantasia), bem como o endereço do restaurante. Ambos os Divs são encapsulados por outro e a função retorna uma lista de dicionários, com um para cada restaurante tal que cada dicionário tem: {‘label’ : o Div descrito acima, ‘value’ : o id do bar a qual a opção se refere, ‘title’ : o endereço do restaurante, ‘search’ : composição do nome, nome fantasia e endereço do restaurante}. Imagem ilustrativa abaixo.



Adicionalmente, ao se selecionar um restaurante, um marcador verde é colocado sobre sua posição. Passando o mouse sobre o marcador pode-se ver o nome fantasia do restaurante caso exista ou o nome legal caso contrário:



Nota-se que o botão de “Buscar” fica travado até que se selecione um restaurante e escolha uma distância máxima de busca. Ao clicar no botão, uma busca pela Trie é feita como discutido anteriormente e os ids dos motoristas hábeis são passados de volta para o frontend e são então calculados os tempos esperados de chegada deles. Opcionalmente, o usuário pode escolher um tempo máximo de chegada. Os motoristas que não podem chegar à tempo, porém ainda na área requisitada continuarão a ser exibidos como marcadores vermelhos, porém, ao pairar o mouse sobre seus marcadores o usuário será avisado de que eles não chegarão à tempo. Esta opção tem o propósito principal de limitar o tempo de busca no grafo e pode ser deixada em branco caso queira filtrar apenas por distância. Desta forma, todos os tempos estimados de chegada serão permitidos.



Motorista dentro da área e com tempo hábil.



Motorista dentro da área sem tempo hábil.

Os demais motoristas, cujos ids que não são recebidos são colocados em uma lista dos motoristas não hábeis. Ao final da execução da subrotina, são retornadas uma lista com os marcadores dos motoristas hábeis (que são mostrados em vermelho) e uma lista com os marcadores dos motoristas não hábeis, que são mostrados em cinza, conforme as imagens acima.

3. Conclusão

De modo geral, o presente trabalho foi uma boa maneira de explorar alguns conceitos vistos em sala; em específico as tries compactas e as Kd-Trees que, apesar de não terem sido mencionadas por nome durante esta documentação, são o conceito no qual se baseiam os geohashes.

Foi também uma maneira de conhecer um pouco mais acerca da criação de páginas web, área em que pessoalmente eu tenho pouca experiência, portanto fez-se necessário aprender alguns conceitos, como a noção de callbacks e como os valores são passados entre eles, bem como revisar outros, como a criação de páginas HTML em geral.